K L NIKIL KUMAR REDDY

SRN: -PES1PG25CA102

Experiential Learning Component-1

1: SIZE OF DATA TYPES

Objective: Write a program to determine and print the sizes of different data types in C.

CODE:

```
#include<stdio.h>
int main(){
int myint;
float myfloat;
double mydouble;
char mychar;
long mylong;
printf("size of intiger:%zu bytes\n",sizeof(myint));
printf("size of float:%zu bytes\n",sizeof(myfloat));
printf("size of double:%zu bytes\n",sizeof(mydouble));
printf("size of character:%zu bytes\n",sizeof(mychar));
printf("size of long:%zu bytes\n",sizeof(mylong));
return 0;
}
```

OUTPUT:

```
size of intiger:4 bytes
size of float:4 bytes
size of double:8 bytes
size of character:1 bytes
size of long:8 bytes
```

2.POINTERS AND ARRAYS

Objective: Write a program to demonstrate pointer arithmetic with a 1-dimensional array.

CODE:

```
#include<stdio.h>
int main(){
   int arr[5]={10,20,30,40,50};
   int *ptr=arr;
   for (int i=0; i<5; i++) {
      printf("%d ",*(ptr+i));
   }
   return 0;
}
```

OUTPUT:

Array elements:10 20 30 40 50

3.DIMENSIONAL ARRAYS

Objective: Create a program that manipulates a 2-dimensional array.

CODE:

```
#include<stdio.h>
int submatrix(int r,int c,int matrix[r][c]){
   int sum=0;
   for (int i = 0;i < r;i++){
      for (int j = 0;j < c;j++){
         sum = sum + matrix[i][j];
      }
   }
   return sum;
}
```

```
int main(){
   int matrix[3][3]={
      {1,2,3},
      {4,5,6},
      {7,8,9}
   };
   int total=submatrix(3,3,matrix);
   printf("sum of matrix is: %d \n",total);
   return 0;
}
```

OUTPUT:

sum of matrix is: 45

4.CALCULATING ADDRESS IN ARRAYS

Objective: Create a program that calculates the address of a specific element in both 1-dimensional and 2-dimensional arrays using their respective formulas.

1-DIMENSIONAL ARRAY:

CODE:

```
#include <stdio.h>
void calculateAddress1D(int baseAddress, int index, int sizeOfElement) {
   int address = baseAddress + (index * sizeOfElement);
   printf("Address of element at index %d in 1D Array: %d\n", index, address);
}
int main() {
   int arr[5] = {10, 20, 30, 40, 50};
   int baseAddress = 2000;
   int sizeOfElement = sizeof(arr[0]);
   int index;
   printf("Enter the index (0–4) of the element: ");
```

```
    scanf("%d", &index);
    if (index < 0 || index >= 5) {
        printf("Invalid index! Please enter between 0 and 4.\n");
    } else {
        calculateAddress1D(baseAddress, index, sizeOfElement);
    }
    return 0;
}
```

OUTPUT:

```
Enter the index (0–4) of the element: 3
Address of element at index 3 in 1D Array: 2012
```

2-DIMENSIONAL ARRAY:

CODE:

```
#include <stdio.h>
void calculateAddress2D(int baseAddress, int row, int col, int totalCols, int sizeOfElement) {
    int address = baseAddress + ((row * totalCols) + col) * sizeOfElement;
    printf("Address of element at (%d, %d) in 2D Array: %d\n", row, col, address);
}
int main() {
    int arr[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int baseAddress = 3036;
    int totalCols = 3;
    int sizeOfElement = sizeof(arr[0][0]);
    int row = 1, col = 2;
```

```
    calculateAddress2D(baseAddress, row, col, totalCols, sizeOfElement);
    return 0;
}
```

OUTPUT:

Address of element at (1, 2) in 2D Array: 3056

5. ARRAYS AS PARAMETERS

Objective: Write a program that uses arrays as function parameters.

CODE:

```
#include <stdio.h>
float calculateAverage(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (float)sum / size;
}
int main() {
    int arr[] = {10, 20, 30, 40};
    int size = sizeof(arr) / sizeof(arr[0]);
    float avg = calculateAverage(arr, size);
    printf("Average of the array: %.1f\n", avg);
    return 0;
}
```

OUTPUT:

Average of the array: 25.0

6. STRUCTURES AND ARRAYS OF STRUCTURES

Objective: Create a program that demonstrates the use of structures and passing them as parameters.

CODE:

```
#include <stdio.h>
struct Student {
   char name[50];
   int age;
   float marks;
};
float calculateAverage(struct Student students[], int size) {
   float sum = 0;
   for (int i = 0; i < size; i++) {
      sum += students[i].marks;
   }
   return sum / size;
}
int main() {
   struct Student students[3] = {
      {"Alice", 20, 85.0},
      {"Bob", 19, 75.5},
      {"Charlie", 21, 75.0}
   };
   int size = 3;
   float average = calculateAverage(students, size);
   printf("Average marks of students: %.1f\n", average);
   return 0;}
```

OUTPUT:

```
Average marks of students: 78.5
```